Relatório – CardioIA | Fase 7
Aluno: Renan de Oliveira Mendes – RM563145

1. Introdução e objetivo da fase

O principal objetivo dessa fase foi concluir o desenvolvimento do CardioIA, integrando todos os módulos das fases anteriores em uma única plataforma funcional.

A proposta foi transformar os projetos de cada fase em uma solução integrada, capaz de receber dados de sensores, processar informações por meio de regras e modelos de inteligência artificial e apresentar os resultados em interfaces web e em um app mobile.

O desenvolvimento foi feito para entregar uma aplicação funcional, focando na integração entre os componentes, experiência do usuário, segurança e demonstração do projeto.

A arquitetura final contempla cinco pontos principais:

1. Sensor: ESP32 simulado no Wokwi, responsável pela captura dos sinais.
2. MicroPython: processamento inicial dos dados diretamente no dispositivo.
3. Backend Python: responsável pela integração dos módulos, autenticação, processamento e persistência.
4. Inteligência Artificial: integração dos modelos e funcionalidades desenvolvidos nas fases anteriores.
5. Interfaces Web e Mobile: exibição das informações e funcionalidades para os usuários.

Um ponto importante do projeto é que os dados utilizados são sintéticos e possuem finalidade apenas acadêmica.

2. Arquitetura e integração da plataforma

O principal trabalho desta fase foi desenvolver o backend como centro da solução.

Em vez de manter cada fase funcionando de maneira separada, foi criada uma estrutura/ programa central em Python com FastAPI, responsável por conectar os módulos existentes e disponibilizar suas funcionalidades por meio de uma API.

Essa abordagem permitiu manter as entregas anteriores, sem modificar diretamente seus códigos, mantendo a possibilidade de melhorias independentes.

2.1 Integração dos módulos anteriores

Cada fase recebeu uma estratégia específica de integração:

Fase	Funcionalidade	Estratégia de integração
Fase 3	Regras de telemetria e IoT	Reimplementação das regras no backend.
Fase 4	Visão computacional	Comunicação HTTP com um serviço separado, isolando o TensorFlow do backend principal.

Fase 5	Assistente conversacional	Provedor local como padrão, com possibilidade de utilização de provedor externo.
Fase 6	Modelo de risco e protocolos	Importação controlada dos módulos Python, com carregamento tardio do modelo.

Uma decisão importante foi rodar de forma local o assistente conversacional. Dessa forma, a funcionalidade pode ser demonstrada sem depender de credenciais ou serviços externos.

No caso da visão computacional, a execução foi isolada em outro serviço. Isso evita que dependências mais pesadas, como TensorFlow, interfiram diretamente na inicialização do backend principal.

2.2 Tecnologias utilizadas

Camada	**Tecnologias**
Backend	Python, FastAPI
Banco de dados	SQLite, SQLAlchemy
Autenticação	JWT, bcrypt
IoT	ESP32, MicroPython, Wokwi
Frontend Web	React, Vite
Frontend Mobile	React Native, Expo
Inteligência Artificial	Modelos e módulos das Fases 1,2, 3, 4, 5 e 6

O backend também passou a utilizar um contrato único de resposta, chamado RespostaAPI, padronizando o retorno da aplicação.

Esse contrato atende status, dados, avisos, limitações e erros, facilitando o tratamento das respostas pelas interfaces.

3. Integração com IoT e fluxo de dados

Outro ponto importante foi integrar a simulação de hardware ao backend, permitindo demonstrar o funcionamento da plataforma da captura dos sinais até a exibição dos resultados.

Para isso, foi utilizado um ESP32 simulado no Wokwi, com os seguintes componentes:

- DHT22: responsável pela leitura simulada da temperatura corporal.
- Botão: utilizado para simular os batimentos cardíacos.
- OLED (SSD1306): apresenta os valores e alertas diretamente no dispositivo.
- LED: sinaliza situações de alerta.

O código em MicroPython realiza uma avaliação inicial dos sinais diretamente no ESP32. Foram implementadas duas condições principais:

- Temperatura superior a 38 °C.
- Frequência cardíaca superior a 120 BPM.

Quando alguma dessas condições é identificada, o dispositivo apresenta o alerta no OLED e aciona o LED.

Após essa avaliação, os dados são enviados ao backend por meio de uma requisição HTTP POST para o endpoint:

/api/v1/iot/leituras

A comunicação utiliza o cabeçalho X-Device-Key para autenticação do dispositivo.

3.1 Fluxo completo da aplicação

O fluxo desenvolvido funciona da seguinte maneira:

1. O ESP32 realiza a leitura da temperatura e recebe os batimentos simulados pelo botão.
2. O MicroPython aplica as regras de avaliação local e apresenta os alertas.
3. Os dados são enviados ao backend por HTTP.
4. O backend recebe a leitura, registra o horário de recebimento e executa novamente as regras de telemetria.
5. A leitura é armazenada no banco de dados e os alertas são devolvidos na resposta da API.
6. Quando uma avaliação de risco é registrada, o backend executa o pipeline da Fase 6, calculando a probabilidade de risco e selecionando os protocolos aplicáveis.
7. As interfaces Web e Mobile consomem os resultados e apresentam as informações ao usuário.

Com isso, foi possível conectar o hardware simulado, as regras de negócio, os modelos de IA e as interfaces em um único fluxo.

4. Segurança e controle de acesso

A segurança foi uma das preocupações durante a construção da solução, principalmente por se tratar de uma aplicação que trabalha com informações relacionadas à saúde.

Foi implementado um sistema de autenticação utilizando JWT, com controle de acesso baseado em papéis (RBAC).

Foram definidos três perfis de usuário:

Perfil	**Permissões**
MEDICO	Leitura e escrita de dados clínicos, assistente e visão computacional.
ADMIN	Apenas leitura clínica.
OPERADOR	Acesso à telemetria e ao assistente.

As permissões são verificadas diretamente no servidor. Portanto, não basta ocultar funcionalidades na interface: o próprio backend valida se o usuário possui autorização para executar cada operação.

Outro detalhe importante é que o papel do usuário é conferido no banco de dados a cada requisição. Isso significa que uma alteração de permissão pode invalidar os acessos concedidos anteriormente.

4.1 Proteção das credenciais

Também foram implementadas algumas medidas para proteger as informações de autenticação:

- Senhas armazenadas utilizando hash bcrypt, com custo 12.
- Segredos configurados exclusivamente por variáveis de ambiente.
- Bloqueio da inicialização do serviço quando os segredos obrigatórios não estão configurados.
- Sanitização das credenciais utilizadas nas fases anteriores antes da publicação do repositório.

Essas medidas ajudam a evitar o vazamento de informações sensíveis e tornam a aplicação mais robusta.

5. Desenvolvimento das interfaces e experiência do usuário

A plataforma possui duas interfaces principais, web e mobile, que consomem o mesmo backend e compartilham o contrato de comunicação da API.

5.1 Interface Web

A interface Web foi desenvolvida utilizando React e Vite.

Ela concentra as principais funcionalidades da plataforma:

- Dashboard com indicadores.
- Avaliação de risco cardíaco.
- Histórico de avaliações.
- Monitoramento das informações.
- Assistente conversacional.

A arquitetura também contempla a publicação da aplicação na Vercel, com integração ao Git para o processo de CI/CD.

5.2 Interface Mobile

A versão Mobile foi desenvolvida utilizando React Native e Expo.

O objetivo foi disponibilizar uma experiência mais direta, com acesso às principais funcionalidades da plataforma:

- Login e autenticação.

- Visualização dos indicadores cardíacos.
- Acesso ao assistente conversacional.

A aplicação foi estruturada para distribuição por meio de APK, utilizando o ecossistema Expo.

5.3 Cuidados com a apresentação dos dados

Um dos pontos que recebeu atenção durante o desenvolvimento foi evitar que informações incompletas ou simuladas fossem apresentadas como medições reais.

Foram adotadas algumas regras:

- Campos sem informação são apresentados como --, em vez de receberem automaticamente o valor zero.
- Informações que o modelo não devolve não são inventadas ou preenchidas pela interface.
- Os resultados preditivos são apresentados separadamente dos dados recebidos pelo dispositivo.

Também foi definido que avisos, limitações, justificativas e detalhes de execução ficam disponíveis de maneira recolhida, podendo ser consultados quando necessário.

Por outro lado, os indicadores e protocolos permanecem visíveis, pois representam informações importantes para a utilização da plataforma.

Essa organização deixa a interface mais objetiva, sem tirar informações relevantes para auditoria e entendimento dos resultados.

6. Testes e validação

Após a implementação das integrações, foram realizados testes automatizados e verificações ponta a ponta para validar o funcionamento dos principais componentes.

Os resultados obtidos foram:

Verificação	**Resultado**
Testes automatizados do backend	659 testes aprovados
Testes de integração ponta a ponta	75 verificações, 0 falhas
Teste das regras da Fase 3	Alerta FEBRE DETECTADA para temperatura de 38,9 °C
Indisponibilidade de serviço externo	Retorno HTTP 502 com mensagem de erro
Build da interface Web	46 módulos, 285 KB (87 KB comprimidos)
Análise estática do frontend	0 erros e 0 avisos

Os testes de ponta a ponta foram distribuídos entre:

- 33 verificações para MEDICO.
- 23 verificações para OPERADOR.
- 19 verificações para ADMIN.

Também foi validada a execução real do pipeline da Fase 6, demonstrando que o modelo foi carregado e executado pelo backend integrado, em vez de simplesmente retornar um resultado simulado.

Esses resultados demonstram que a integração entre os módulos foi validada em diferentes cenários, incluindo autenticação, autorização, persistência, comunicação entre serviços e processamento das informações.

7. Observações

Existem algumas limitações importantes que precisam ser consideradas.

Dados sintéticos e ausência de validação clínica: os dados utilizados possuem finalidade acadêmica. O modelo foi treinado em uma base acadêmica e sua probabilidade representa o episódio assistencial, não um prognóstico individual.

Telemetria: as leituras recebidas são vinculadas ao dispositivo, e não diretamente a um paciente.

Alertas de telemetria: os alertas são derivados das regras de negócio e retornados durante a ingestão, mas não são armazenados como registros independentes no banco de dados.

Frequência cardíaca: os batimentos são simulados por meio de um botão no ESP32. Não existe um sensor físico de pulso no circuito.

Assistente conversacional: sua função é fornecer orientações e encaminhar o usuário para atendimento profissional quando necessário, sem realizar diagnósticos médicos.

Essas limitações foram consideradas durante o desenvolvimento para evitar que a plataforma apresentasse resultados com um nível de confiabilidade superior ao que realmente foi validado.

8. Conclusão

A Fase 7 representou a conclusão da integração dos componentes desenvolvidos ao longo do CardioIA.

O principal resultado foi transformar módulos que anteriormente funcionavam de maneira independente em uma plataforma integrada, com backend centralizado, comunicação com hardware simulado, processamento por inteligência artificial e interfaces Web e Mobile.

Além da integração, foram implementados mecanismos de autenticação, controle de acesso, persistência de dados, padronização das respostas da API e testes automatizados.

Um dos principais diferenciais da solução foi desenvolver uma arquitetura que não depende obrigatoriamente de credenciais de terceiros para demonstrar suas funcionalidades. Os componentes que normalmente exigiriam serviços externos

possuem alternativas locais dentro do próprio projeto, permitindo uma execução mais independente e reproduzível.

Com isso, o MVP final da CardioIA apresenta um fluxo completo: desde a captura dos sinais no dispositivo até o processamento das informações e sua apresentação nas interfaces.

O resultado final é uma plataforma acadêmica integrada e funcional, que demonstra como diferentes tecnologias podem trabalhar em conjunto para construir uma solução de monitoramento e inteligência.